

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
# train_and_save_model.py
import os
import cv2
import numpy as np
from sklearn.utils import shuffle
from skimage.feature import graycomatrix, graycoprops
from skimage import measure
import joblib

from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

import multiprocessing
from tqdm import tqdm

# ----- CONFIG -----
# Set these to your dataset root; keep Training/Testing separated if dataset already provides splits.
data_dir = '/content/drive/MyDrive/MRI Images/'
respect_given_split = True # If Training/Testing are true splits, keep them distinct for evaluation.

# ----- IO SCAN -----
all_paths = []
all_labels = []
split_tags = [] # "train" or "test" if respecting given split

for label in ['Training', 'Testing']:
    folder_path = os.path.join(data_dir, label)
    for tumor_type in os.listdir(folder_path):
        tumor_folder = os.path.join(folder_path, tumor_type)
        if not os.path.isdir(tumor_folder):
            continue
        for image in os.listdir(tumor_folder):
            all_paths.append(os.path.join(tumor_folder, image))
            all_labels.append(tumor_type)
            split_tags.append('train' if label.lower().startswith('train') else 'test')

all_paths, all_labels, split_tags = shuffle(all_paths, all_labels, split_tags, random_state=42)
print(f"Total images found: {len(all_paths)}")

# ----- FEATURE EXTRACTOR -----
def build_gc_mask_from_seed(image_bgr, clean_seed_mask):
    # 0: sure BG, 1: sure FG, 2: prob BG, 3: prob FG
    gc_mask = np.full(image_bgr.shape[:2], cv2.GC_PR_BGD, np.uint8)
    gc_mask[clean_seed_mask == 0] = cv2.GC_BGD
    gc_mask[clean_seed_mask == 1] = cv2.GC_PR_FGD
    core = cv2.erode(clean_seed_mask, np.ones((5,5), np.uint8), iterations=1)
    gc_mask[core == 1] = cv2.GC_FGD
    return gc_mask

def seed_from_otsu(image_gray):
    image_blur = cv2.medianBlur(image_gray, 5)
    _, otsu_mask = cv2.threshold(image_blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    kernel_open = np.ones((5,5), np.uint8)
    mask_open = cv2.morphologyEx(otsu_mask, cv2.MORPH_OPEN, kernel_open)
    kernel_close = np.ones((10,10), np.uint8)
    clean_seed_mask = cv2.morphologyEx(mask_open, cv2.MORPH_CLOSE, kernel_close)
    return clean_seed_mask

def fallback_seed(image_gray):
    # CLAHE + Otsu retry as minimal fallback for heterogeneous MRIs
    image_blur = cv2.medianBlur(image_gray, 5)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    eq = clahe.apply(image_blur)
    _, otsu_mask2 = cv2.threshold(eq, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    mask_open = cv2.morphologyEx(otsu_mask2, cv2.MORPH_OPEN, np.ones((5,5), np.uint8))
    clean_seed_mask = cv2.morphologyEx(mask_open, cv2.MORPH_CLOSE, np.ones((10,10), np.uint8))
    return clean_seed_mask

def grabcut_refine(image_bgr, init_mask):
    gc_mask = build_gc_mask_from_seed(image_bgr, init_mask)
    bgdModel = np.zeros((1,65), np.float64)
    fgdModel = np.zeros((1,65), np.float64)
```

```

fgdModel = np.zeros((1,50), np.float64)
cv2.grabCut(image_bgr, gc_mask, None, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_MASK)
final_mask = np.where((gc_mask == cv2.GC_FGD) | (gc_mask == cv2.GC_PR_FGD), 1, 0).astype('uint8')
return final_mask

def extract_features(image_path):
    try:
        image_bgr = cv2.imread(image_path)
        if image_bgr is None:
            return None
        image_gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

        # Seed
        clean_seed_mask = seed_from_otsu(image_gray)

        contours, _ = cv2.findContours(clean_seed_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        if not contours:
            clean_seed_mask = fallback_seed(image_gray)
            contours, _ = cv2.findContours(clean_seed_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
            if not contours:
                return None

        # GrabCut refinement with mask init
        final_mask = grabcut_refine(image_bgr, (clean_seed_mask > 0).astype(np.uint8))

        # Region props
        labels = measure.label(final_mask)
        props = measure.regionprops(labels, intensity_image=image_gray)
        if not props:
            return None
        tumor_blob = max(props, key=lambda p: p.area)

        # Shape features
        area = tumor_blob.area
        perimeter = tumor_blob.perimeter
        eccentricity = tumor_blob.eccentricity
        solidity = tumor_blob.solidity

        # Texture features (compute GLCM on bounding box crop, masked)
        minr, minc, maxr, maxc = tumor_blob.bbox
        crop_gray = image_gray[minr:maxr, minc:maxc]
        crop_mask = final_mask[minr:maxr, minc:maxc].astype(bool)

        if crop_gray.size < 4 or crop_mask.sum() < 10:
            return None

        # Fill background of crop with local mean to reduce co-occurrence bias
        crop = crop_gray.copy()
        bg_mean = int(crop_gray[crop_mask].mean()) if crop_mask.sum() > 0 else int(crop_gray.mean())
        crop[~crop_mask] = bg_mean

        # Normalize to 8-bit
        crop_u8 = cv2.normalize(crop, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

        # Multi-distance, multi-angle GLCM
        distances = [1, 2, 4]
        angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]
        glcm = graycomatrix(crop_u8, distances=distances, angles=angles, levels=256, symmetric=True, normed=True)

        # Aggregate stats (mean across distances and angles)
        def agg(prop):
            m = graycoprops(glcm, prop)
            return float(m.mean())

        contrast = agg('contrast')
        energy = agg('energy')
        homogeneity = agg('homogeneity')
        correlation = agg('correlation')

        return [area, perimeter, eccentricity, solidity, contrast, energy, homogeneity, correlation]
    except Exception:
        return None

# ----- PARALLEL EXTRACTION -----
num_cores = multiprocessing.cpu_count()
print(f"Starting feature extraction... Using {num_cores} CPU cores in parallel.")

with multiprocessing.Pool(processes=num_cores) as pool:
    results = list(tqdm(pool.map(extract_features, all_paths), total=len(all_paths)))

X = []
y = []
tags = []

```

```

for i, features in enumerate(results):
    if features and len(features) == 8:
        X.append(features)
        y.append(all_labels[i])
        tags.append(split_tags[i])

print(f"Successfully extracted features from {len(X)} images.")

# ----- SPLIT & TRAIN -----
if respect_given_split:
    # Use provided split
    X_train = [f for f, t in zip(X, tags) if t == 'train']
    y_train = [l for l, t in zip(y, tags) if t == 'train']
    X_test = [f for f, t in zip(X, tags) if t == 'test']
    y_test = [l for l, t in zip(y, tags) if t == 'test']
else:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                        random_state=42, stratify=y)

print(f"Training samples: {len(y_train)}")
print(f"Testing samples: {len(y_test)}")

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=300, max_depth=None, random_state=42, n_jobs=-1))
])

print("Training the Random Forest model...")
pipe.fit(X_train, y_train)
print("Model training complete!")

y_pred = pipe.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Save
model_save_path = '/content/drive/MyDrive/tumor_classifier_model.pkl'
joblib.dump(pipe, model_save_path)
print(f"Model saved to: {model_save_path}")

```

```

Total images found: 7023
Starting feature extraction... Using 2 CPU cores in parallel.
100%|██████████| 7023/7023 [00:00<00:00, 2309596.75it/s]
Successfully extracted features from 7023 images.
Training samples: 5712
Testing samples: 1311
Training the Random Forest model...
Model training complete!
Model Accuracy: 91.30%

```

```

Classification Report:

```

	precision	recall	f1-score	support
glioma	0.97	0.74	0.84	300
meningioma	0.81	0.93	0.87	306
notumor	0.98	1.00	0.99	405
pituitary	0.90	0.95	0.92	300
accuracy			0.91	1311
macro avg	0.91	0.91	0.90	1311
weighted avg	0.92	0.91	0.91	1311

```

Model saved to: /content/drive/MyDrive/tumor_classifier_model.pkl

```

